🐳🚀 Building & Running Docker Image from Dockerfile

Running Spring Boot App Inside Docker & Introduction to Docker Compose – Creative Theory Notes

---

1. Running the Docker Image Built from Dockerfile

After creating the Docker image using a **Dockerfile**, the speaker runs the image tagged as **v3**.

Key steps:

- The container is started using `docker run`
- Port **8081** inside the container is exposed to the host
- The container starts successfully

Verification:

- Container status is checked using Docker commands
- Application is accessed via a web browser
- The REST endpoint responds correctly

This confirms the Docker image works as expected.

---

2. Importance of Dockerfile Automation

The key takeaway from this segment is **automation**.

Previously:

- JAR was copied manually
- Container was modified step by step
- Image was committed manually

Now:

- All steps are defined in a **Dockerfile**
- Image creation is done with a single command
- Process is reproducible and consistent

This demonstrates why Dockerfile is essential in professional development workflows.

---

3. Running Spring Boot App Inside Docker Container

The Spring Boot application is now:

- Fully packaged inside a Docker image
- Running independently of the host system
- Accessible through mapped ports

Benefits:

- No need to install Java on the host
- Same behavior across all environments
- Easy sharing and deployment

The example REST application returns **"Hello World"**, confirming successful execution.

---

4. Portability Across Environments

One of Docker's biggest advantages is **portability**.

A Docker image can:

- Run on local machines
- Run on cloud platforms
- Be shared with other developers

As long as Docker is installed, the application behaves exactly the same everywhere.

---

5. Recap of Docker Learning Journey

At this point, the speaker summarizes what has been achieved:

- Pulled Docker images
- Ran containers
- Managed images and containers
- Installed and used JDK inside Docker
- Built a Spring Boot application
- Packaged it as a JAR
- Dockerized the application using Dockerfile
- Successfully ran the app inside a container

This confirms end-to-end understanding of Docker fundamentals.

---

6. Limitation of Single-Container Applications

The current setup involves:

- One container
- One Java Spring Boot application

However, real-world applications usually require:

- Databases
- Message queues
- Caching services

Running everything in a single container is not practical.

---

7. Introduction to Multi-Container Applications

To build real applications, multiple containers are required.

Example:

- One container → Spring Boot application
- One container → Database (PostgreSQL)

Each service runs in its own container but must communicate with others.

---

8. Choosing PostgreSQL as Database

The presenter selects **PostgreSQL** as the relational database.

Key note:

- The approach is **database-agnostic**
- Other databases can be used by changing:
    - Connection URL
    - Username
    - Password

This flexibility is one of Docker's strengths.

---

9. Why Docker Compose Is Needed

Managing multiple containers manually is complex:

- Multiple `docker run` commands
- Port mappings
- Environment variables
- Network configuration

Docker Compose solves this problem.

Docker Compose allows:

- Defining multiple containers in one file
- Managing networks automatically
- Starting all services with one command

---

10. Introduction to Docker Compose Workflow

The speaker introduces Docker Compose using a **step-by-step approach**.

First step:

- Create a new Spring Boot web application
- Add database connectivity (PostgreSQL)
- Use Maven as the build tool
- Use Spring Boot version **3.1.5**

Project name:

- **telescope**

This project will demonstrate **application + database** running together.

---

11. Preparing for Docker Compose Integration

The new project is designed to:

- Connect to a relational database
- Be containerized
- Run alongside PostgreSQL

This sets the foundation for:

- Microservices
- Real-world backend applications
- Scalable architectures

---

12. Conclusion

This section completes the transition from:

- Local development
- To single-container Docker apps
- To **multi-container architecture**

Running a Spring Boot app inside Docker proves the power of containerization. Introducing Docker Compose marks the next stage—**orchestrating multiple services efficiently**.

If you want next, I can:

- Explain **Docker Compose theory**
- Write a **docker-compose.yml for Spring Boot + PostgreSQL**
- Explain **container networking in Compose**
- Show **environment variables for DB config**
- Turn everything into **exam notes / project guide** 🐳📦